

markup2html

Render Markdown (`.md`) and Org (`.org`) documents to standalone, RFC-styled HTML pages.

- `markup2html.mjs` — the renderer and CLI (replaces the former `contrib/markup2html.sh`). Handles conversion (marked for Markdown, batch Emacs for Org from the CLI, in-process Org export from Emacs) and the full HTML assembly. Runs on `bun` or `node`.
- `markup2html.el` — thin Emacs wiring around the same script; all rendering happens in the script.
- `slim-marked.mjs` — maintenance script for the vendored marked.

The package is self-contained: it ships `rfc-style.css` (the base stylesheet, inlined into every page) and a pinned, vendored copy of marked (`slimmed marked.esm.js` — see below). The slimmed variant drops marked functions that `markup2html` never uses: 41.7 KB → 35.7 KB.

Demo

See [readme-sample.pdf](#) or [readme-sample.html](#), to see the result of rendering this file into HTML or PDF (the pdf file is created from the html one).

CLI

```
bun markup2html.mjs [-o OUTPUT] [--css FILE] <input.md | input.org>
# or: node markup2html.mjs ...    (both runtimes work)
```

Writes the page to **stdout**; `-o OUTPUT` writes it to a file instead (`-o -` is the same as `stdout`).
`--css` replaces the packaged stylesheet. Requirements: `bun` or `node` on `PATH`, and `emacs` on `PATH` for `.org` inputs.

The `--html` mode

In `--html` mode the script skips markdown/org conversion entirely: it reads a ready-made HTML **body fragment** from `stdin` and wraps it in the page shell — `<head>` with the inlined stylesheet and CDN layer, the title, the meta table, the `<main class="rfc markdown-body">` wrapper, and the same conditional enhancers as file mode (mermaid loader, highlight.js, floating TOC — each only when the body contains one).

```
markup2html.mjs --html [--title TITLE] [--meta "KEY: VAL"]... [--h1] \
  [-o OUTPUT] [--css FILE] < body.html
```

Behavior:

- The input is a body **fragment**, not a complete document — no `<html>`, `<head>` or `<body>`; the shell provides them.
- `--title` sets the `<title>` (HTML-escaped). Without it the title is empty.
- `--h1` additionally emits the title as the page's `<h1>`. Org bodies from `org-export-as` don't contain a heading, so `markup2html.el` passes this flag; when the fragment already carries its own `<h1>`, omit it.
- Each `--meta "KEY: VAL"` adds one row to the key/value table at the top of the page (the first colon splits; leading spaces of the value are stripped). Values are inserted as-is apart from HTML escaping — FILETAGS-style `:a:b:` values must be transformed before passing.
- The enhancer scan runs on the fragment: `language-mermaid` / `src-mermaid` in the body pulls in the mermaid loader, a `<code class="language-...">` block pulls in `highlight.js`, and an `h2` – `h4` pulls in the floating TOC. A fragment without any of these produces a script-free page.
- Without `-o` the page goes to stdout; `-o -` is the same, so the mode doubles as a `stdin→stdout` filter. `--css` swaps the inlined stylesheet.

Who uses it:

- **markup2html.el** — exports the Org buffer in-process (`org-export-as`, `body-only`, ~5 ms) and pipes it here with `--title`, `--h1` and one `--meta` per leading `#+KEYWORD:` line. One bun process instead of a batch-Emacs roundtrip (~25 ms vs ~300 ms per export).
- Anything else that produces an HTML fragment and wants the RFC shell — the bytes are identical to what the file modes emit for the same document.

Emacs

Requires Emacs 30.1+ (the `Elisp` module uses modern APIs like `string-lines` and `string-replace`).

```
(use-package markup2html
  :load-path "/path/to/markup2html")
```

Command	What it does
<code>markup2html-export</code>	Render the visited file, write <code><stem>.html</code>
<code>markup2html-export-and-preview</code>	Export, then open the result in a browser
<code>markup2html-preview-exported</code>	Open the exported <code><stem>.html</code> (error if absent)
<code>markup2html-command</code>	<code>markdown-command -compatible</code> hook

All commands work in `markdown-mode`, `gfm-mode`, `markdown-ts-mode` and `org-mode` buffers alike. Markdown buffers are rendered as a whole from the visited file (region arguments are ignored; save first to include your edits); Org buffers are exported in-process, unsaved edits included.

markdown-mode integration

- **Runtime selection:** `markup2html-js-runtime` — `'bun` (default) or `'node` (set via `setopt`).
- **Render and preview** route `markdown-mode`'s native commands through `markup2html`:

```
(setopt markdown-command #'markup2html-command)
```

With that, `C-c C-c m` compiles into the `*Markdown output*` buffer (the page is streamed from the renderer's stdout — no `.html` file is written next to the source), `C-c C-c p` previews in the browser, and `C-c C-c e` writes the HTML file next to the source.

- **Open the rendered page externally** on demand, mirroring `markdown-mode`'s `markdown-open-command` (bound to `C-c C-c o`):

```
(setopt markdown-open-command
  (lambda ()
    (browse-url-of-file (markup2html-export))))
```

This repo's `.dir-locals.el` already sets `markdown-command` buffer-locally for `.md` files.

- **Org integration:** Org buffers are exported to body-only HTML **in-process** (`org-export-as`, ~5 ms in a loaded session, unsaved edits included) and piped to the script's stdin (`--html` assembly mode) — no batch-Emacs roundtrip, roughly 25 ms total vs ~300 ms for spawning a throwaway emacs per export. The export dispatcher entry `C-c C-c e R` (`r` = write the file, `o` = write and open) runs the same commands.

Output

Standalone page with the base stylesheet inlined; github-markdown-css and the highlight.js theme layer on top via CDN links pinned with SRI hashes (no theme switching; offline the page keeps the base look). Pages with mermaid diagrams, code fences or sections load the corresponding client-side enhancers; everything else stays script-free. Documents with YAML front matter (md) or leading `#+KEYWORD:` lines (org) render it as a key/value table; documents with sections get a floating table of contents, open by default.

Upgrading pinned assets

- `marked` : download the new `lib/marked.esm.js` over the vendored `marked.esm.js` , then run `bun slim-marked.mjs marked.esm.js` . Every cut in the slim script asserts its markers, so an upstream restructure fails loudly instead of silently corrupting the file; re-render a document and diff to confirm the output is still what you expect.
- CDN assets (github-markdown-css, highlight.js, mermaid): bump the version and its SRI hash together in `markup2html.mjs` (`curl -s URL | openssl dgst -sha384 -binary | openssl base64 -A`).

Samples

```
function bubbleSort(numbers: number[]): number[] {
  // Create a copy to avoid mutating the original array
  const arr = [...numbers];
  const n = arr.length;

  for (let i = 0; i < n - 1; i++) {
    let swapped = false;

    for (let j = 0; j < n - 1 - i; j++) {
      if (arr[j] > arr[j + 1]) {
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
        swapped = true;
      }
    }
  }

  // Stop early if no swaps occurred in this pass
  if (!swapped) break;
}

return arr;
}
```

